

AWSKRUG · 마곡 지역소모임

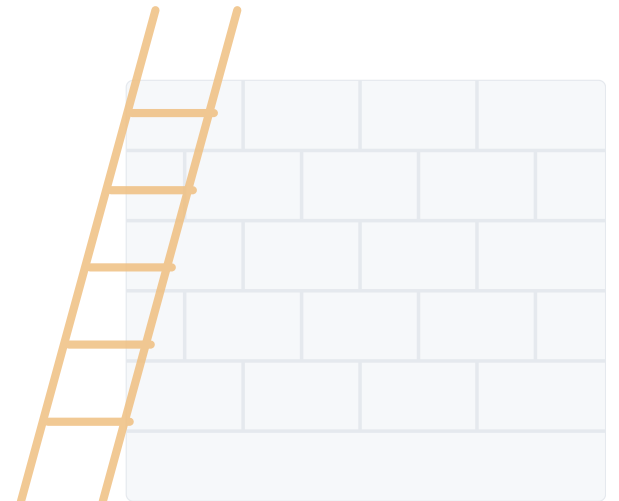
AWS Bedrock을 활용한 LLM in the test loop

테스팅 루프에 LLM을 안전하게 끼우는 법 — 그리고 그 경계

● 백창준

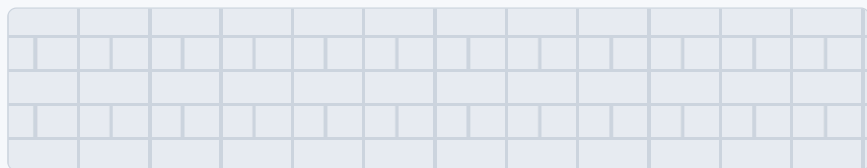
LG유플러스 CTO 품질PMO팀

20 min



도입 · 앞 발표에서 이어서

가드레일이 막는 것, 못 막는 것



가드레일

코드로 규칙을 적으면, **결정적으로** 막힌다.

if (violation) → deny — 매번 같은 결과



판단이 필요한 영역

규칙으로 적을 수 없고, **맥락을 읽어야** 갈린다.

여기엔 무엇을 넣어야 할까?

BLOCKER	S6418	Make sure this "aws_secret" is not a hard-coded secret.	PaymentConfig.java
BLOCKER	S2095	Use try-with-resources or close this "Connection".	UserService.java
MAJOR	S2259	A "NullPointerException" could be thrown; "payload" is nullable.	OrderHandler.java
CRITICAL	S3776	Refactor this method to reduce its Cognitive Complexity from 31 to 15.	OrderProcessor.java
BLOCKER	S6418	Make sure this "stripe_secret" is not a hard-coded secret.	PaymentGatewayIT.java
CRITICAL	S2447	Do not return null from this "Boolean" method.	UserService.java
MAJOR	S1192	Define a constant instead of duplicating this literal 7 times.	UserService.java
MAJOR	S106	Replace this use of System.out by a logger.	OrderProcessor.java
MINOR	S1481	Remove this unused "tmp" local variable.	ReportScheduler.java
MAJOR	S166	Remove this use of "hardcoded" private	AuthFilter.java

도입 · 질문 하나

이 중 진짜 결함은 몇 개일까요?

자동화의 벽을 **코드로** 뚫어왔습니다

우회

인증의 벽 — SMS · OTP · 패스키

자동화를 가로막는 인증 단계를 우회·돌파.
OTP 메일을 테스트가 직접 읽어 통과한다.

github.com/baekchangjoon/MailosaurDemo

직접 제작

도구의 벽 — 커스텀 브라우저

상용 자동화 툴이 너무 비싸서,
CDP 기반 자동화 도구를 직접 만들어 돌파.

github.com/baekchangjoon/CEF-Browser · [-Playwright](#)

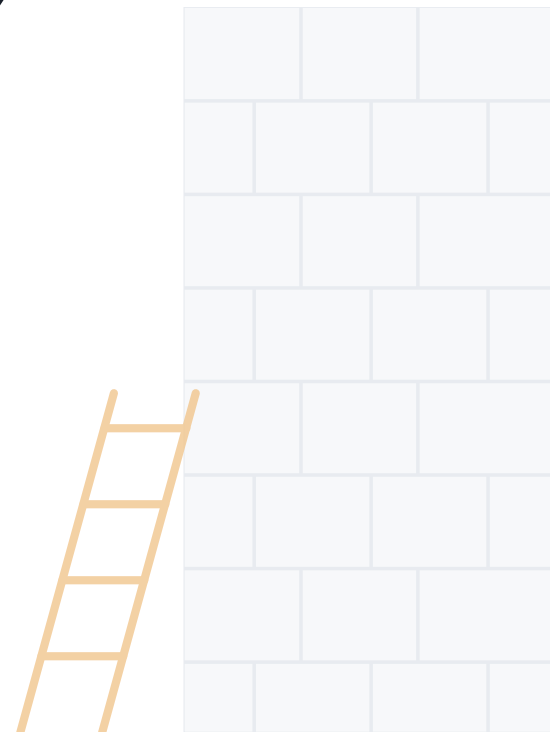
"방법은 알았다. 구현이 어려웠을 뿐." → **코드로 해결**

전환

그런데, 코드로 방법을 적을 수 없는 벽이 있습니다

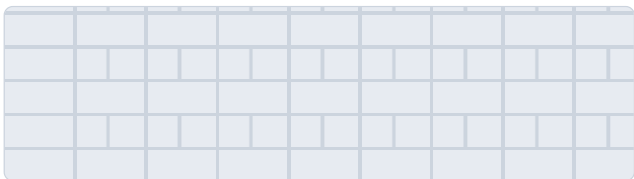
"이 경고가 진짜인가, 헛것인가" — 판단.

사람이 봐야 했고, 자동화는 거기서 멈췄다.



멘탈모델

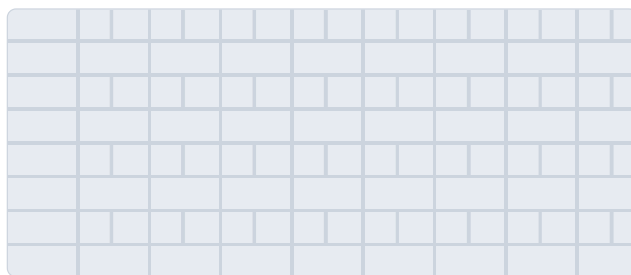
벽은 두 종류, 그리고 경계



① 구현의 벽

방법은 안다,
구현이 어려울 뿐.

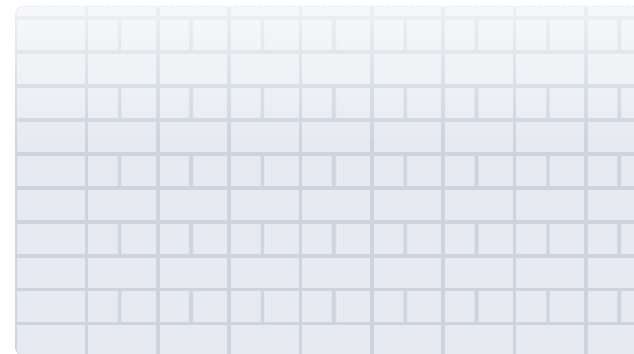
코드로 뚫는다 — 결정적



② 판단의 벽

사람은 할 수 있다 —
다만 **사람**이 병목이다.

사람을 풀어주면 풀린다 — LLM



③ 불가능의 벽

정답을 **사람도** 못 정하는 곳.

예: "알아서 잘 동작하게" 티켓 — 통과 기준을 **사람도** 못 정한다

LLM도 못 뚫는다

핵심 질문 사람이 병목인가, 사람도 **불가능**인가 — 그것부터 구분하라

정량은 코드로, 정성은 LLM으로

코드로

결정적이어야 할 테스트

고정 동작 · 정량 검증

방법을 적을 수 있는, 쉬운 문제

LLM으로

비결정적 — 테스트와 분리해서

판단 · 정성 평가

규칙으로 못 적는, 난해한 문제

사람이 병목이면 → LLM이 푼다

사람도 불가능하면 → LLM도 못 푼다

"그냥 룰로 거르면 되잖아?"

맞는 말 같다. 저도 그렇게 시작했다.

RULE 1 test 경로 제외 — path contains "/test/" → 오탐 처리

RULE 2 시크릿 키워드 매칭 — secret | key | token | credential | passw

RULE 3 Shannon 엔트로피 임계값 — $\text{entropy}(\text{string}) \geq 3.0$ → 실결함 의심

그런데 **두 개의 사례**에서 박살이 났다

데모 · 경계 쌍

같은 규칙 **S6418**, 정반대 정답

```
// PaymentConfig.java · src/main/java/...  
// AWS 공식 문서에 공개된 예제 키  
static final String AWS_ACCESS_KEY_ID =  
    "AKIAIOSFODNN7EXAMPLE";
```

≠

```
// PaymentGatewayIT.java · src/test/java/...  
// "임시로" 실 게이트웨이를 때리는 테스트  
private static final String STRIPE_SECRET =  
    "sk_live_51Hx..."; // 운영 키
```

룰 → **경고 (허경보)** — 키워드·엔트로피 매칭

정답 → **오탐** — 구조·엔트로피는 실키와 동일

룰 → **통과 (놓침)** — "test 경로 제외"가 삼킴

정답 → **실결함** — 진짜 운영 키 유출

룰을 한쪽에 맞추면 반대쪽이 깨진다 — 둘을 가르는 건 의도와 맥락뿐

데모 · 결과

규칙 기반

0/3

LLM (Bedrock · Claude)

3/3

사례	정답	규칙	LLM
A. AWS 예제 키 · main 경로 S6418	오탐	×	✓
B. sk_live_ 운영 키 · test 경로 S6418	실결함	×	✓
C. NPE 경고 · 윗줄에 커스텀 검증기 S2259	오탐	×	✓

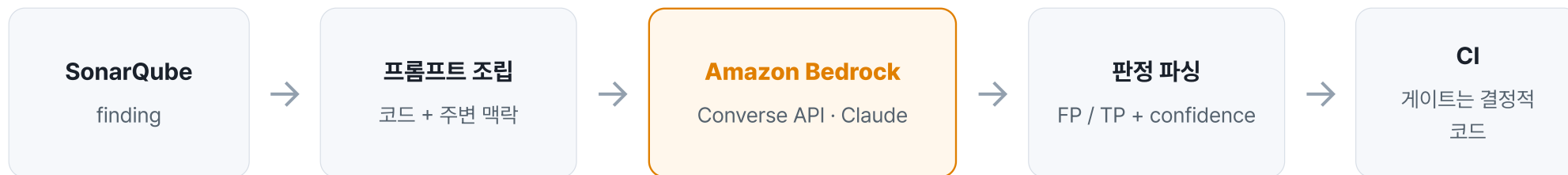
```
$ java FalsePositiveTriageDemo.java — 의존성 0 · 네트워크 불필요 · --live 재현 가능
```

표본 3건은 대표 예시 — fp-corpus **32건 전체에서 재현율 94%** (judge: opus · 상세는 비용 슬라이드)

이 판단, 사람도 한다 — 수천 개를 못 볼 뿐. 사람이 병목이었고, LLM이 그 병목을 풀었다.

데모 · 구현

어떻게 만들었나 — 전부 AWS



● fixer / judge 분리 — 역할별 모델 라우팅

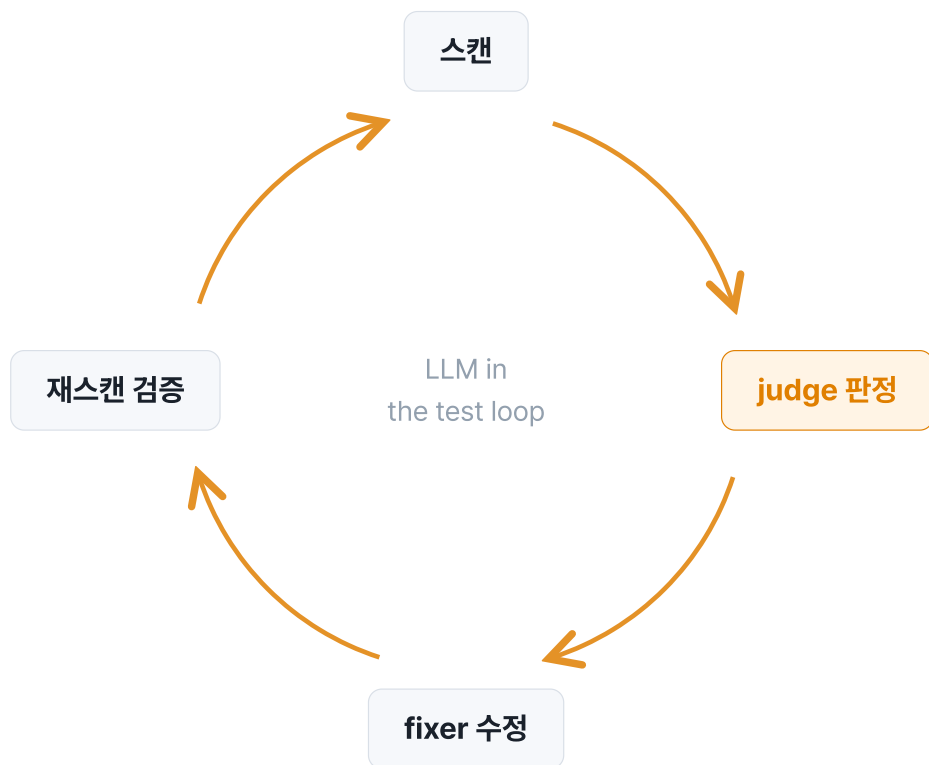
수정 = 에이전트 CLI (파일 편집)

판정 = Bedrock Converse (편집 불필요)

LLM-agnostic — claude · gemini · kiro CLI

데모 · 루프

단발이 아니라 루프



LLM은 게이트가 아니라 분류기

judge가 오탐을 걸러내고, fixer는 진짜 결함만 수정.
최종 판정은 결정적 코드가 한다.

애매하면 보수적 풀백

판정 불능 응답 → "실결함" 처리.
놓치는 것보다 헛경보가 안전하다.

검증은 재스캔으로

이슈가 서버에서 실제로 닫혔을 때만 수정으로 집계.
LLM의 자기 보고는 신뢰하지 않는다.

judge 자체도 검증 대상 진행 중

재스캔은 fixer만 검증한다. judge 판정은
사람 라벨 기준셋과의 일치율로 캘리브레이션.

효과

못 하던 걸 하게 됐다 — 핫스팟 분석

$$\text{인지 복잡도 (AST)} \times \text{최신성 감쇠 (Recency Decay)} \times \text{커버리지 배수 } 1/(\text{cov}+0.1) = \text{Composite Hotspot Score}$$

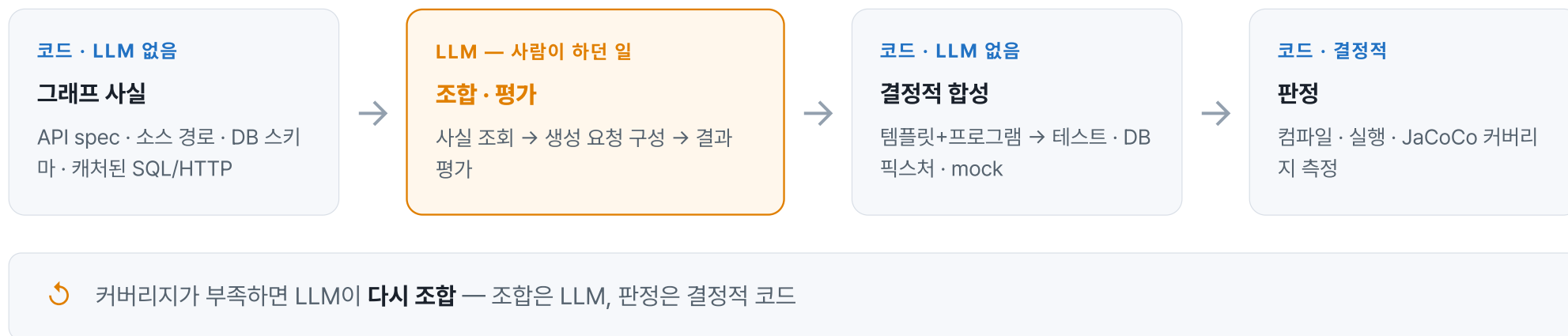


50만 줄에서 테스트 노력을 정량적으로 배분한다

효과 · 같은 원칙의 다음 적용

테스트 자산 생성 — 사람 몫은 **조합**뿐이었다

API 스펙·소스·DB 스키마는 코드에 있고, 템플릿 합성도 코드가 한다.



비용

비용도 설계의 일부

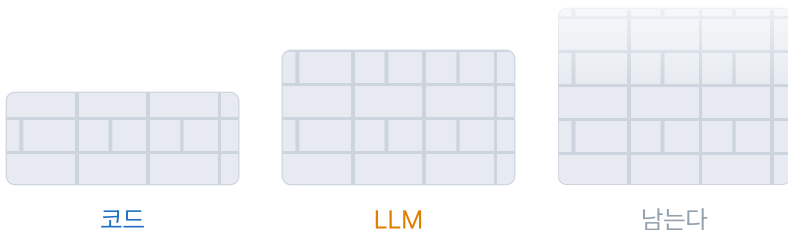
모델 세트 (수정 / 판정)	오탐 검출 (32개 중)	재현율	1 run 비용
opus · opus	30	94%	\$0.57
sonnet · opus	28	88%	\$0.60
haiku · sonnet	24	75%	\$0.40

가장 싼 세트가 최선이 아니다 — 더 똑똑한 judge가 전체 비용을 낮춘다

놓친 오탐 1건의 진짜 비용 = 멀쩡한 코드 변경 + 그걸 걸러내는 사람의 리뷰 시간

전략도 같은 결론 — triage_review (judge 판정 → fixer 수정 → judge 리뷰, 3패스)가 재현율·정밀도 모두 최고 — 운영 기본값

진행 중 측정 한계 — 이 코퍼스는 **전부 오탐**이라 재현율만 측정됨. 실결함을 오탐으로 놓치는 비율은 **실결함 혼합 코퍼스** 벤치마크로 측정 중



구현의 벽은 엔지니어링으로, 판단의 벽은 LLM으로.

사람조차 못 뚫는 벽은 — 남는다. ("알아서 잘 동작하게"의 통과 기준처럼)

여러분 파이프라인의 어디가 **판정**이고 어디가 **판단**인지 그려보세요. 판단에만, 시험적으로 LLM을.

github.com/baekchangjoon/sonarqube-ai-agent

github.com/baekchangjoon/hotspot-analysis

코드 · 기법은 레포/블로그에 공개